

UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC

CAMPUS Prof. SOANE NAZARÉ DE ANDRADE

VINÍCIUS NUNES DE CARVALHO

INTEGRATION HUB - SIMPLIFICANDO A INTEGRAÇÃO ENTRE APIs

ILHÉUS - BAHIA

2024

VINÍCIUS NUNES DE CARVALHO

INTEGRATION HUB - SIMPLIFICANDO A INTEGRAÇÃO ENTRE APIs

Relatório apresentado à Universidade Estadual de Santa Cruz, como parte das exigências para a obtenção do título de Bacharel em Ciência da Computação

Área de Concentração: Ciência da Computação

Orientador: Helder Conceição Almeida

ILHÉUS - BAHIA

2024

VINÍCIUS NUNES DE CARVALHO

INTEGRATION HUB - SIMPLIFICANDO A INTEGRAÇÃO ENTRE APIs

Esta Monografia foi julgada adequada para a obtenção do título de Bacharel em Ciência de Computação, sendo aprovada em sua forma final pela banca examinadora:

Ilhéus, 05 de dezembro de 2024

_____

Prof. Me. Helder Conceição Almeida
UESC/DCET
(Orientador)

_____

Prof. Dra. Martha Ximena Torres Delgado
UESC/DCET

_____

Prof. Me. Otacílio José Pereira
UESC/DCET

AGRADECIMENTOS

À Deus

À meu pai, minha mãe e meu irmão

À meus avós, meus tios, meus primos

Aos meus amigos

Ao CEPEDI por proporcionar um ambiente propício para o meu desenvolvimento profissional

À UESC por proporcionar toda a estrutura necessário para a graduação

INTEGRATION HUB - SIMPLIFICANDO A INTEGRAÇÃO ENTRE APIs

RESUMO

Este trabalho descreve o desenvolvimento do “IntegrationHub”, plataforma que visa simplificar a integração das APIs da empresa CEPEDI. Dada a demanda contínua por mais soluções de interoperabilidade entre sistemas, o IntegrationHub foi projetado para centralizar e conectar automaticamente APIs como triagem de CEP, CNPJ e bancos, a fim de criar um ambiente mais seguro e rápido no desenvolvimento de software. O desenvolvimento do projeto, realizado através da metodologia ágil Scrum, foi desenvolvido através de sprints de etapas de funcionalidades essenciais definidas, implementadas e revisadas, bem como etapas de testes para garantir que os sistemas estivessem estáveis e eficientes. A arquitetura do sistema foi projetada em três camadas, com o software .NET e as ferramentas PostgreSQL e DBeaver, visando proporcionar uma estrutura robusta, escalável e preparada para integrações futuras. Como resultado, o Integration Hub permitiu a redução do tempo de desenvolvimento e manutenção, o que contribuiu para o aumento da produtividade.

Palavras-chave: Integração das APIs. Scrum. Três camadas.

INTEGRATION HUB - SIMPLIFYING INTEGRATION BETWEEN APIs

ABSTRACT

This work describes the development of “IntegrationHub”, a platform that aims to simplify the integration of CEPEDI company APIs. Given the continued demand for more cross-system interoperability solutions, IntegrationHub is designed to centralize and automatically connect APIs such as zip code, CNPJ, and bank screening to create a safer, faster software development environment. The development of the project, carried out using the agile Scrum methodology, was developed through sprints of stages of essential functionalities defined, implemented and reviewed, as well as testing stages to ensure that the systems were stable and efficient. The three-tier architecture and .NET software using PostgreSQL and DBeaver tools aimed for a robust and scalable structure, allowing for easier future integrations. Finally, the results in IntegrationHub managed to reduce development and maintenance time, leading to increased productivity and innovation in the corporate environment.

Keywords: API integration. Scrum. Three layers.

LISTA DE ABREVIATURA E SIGLAS

API	Interface de Programação de Aplicação
CEPEDI	Centro de Pesquisa e Desenvolvimento Tecnológico em Informática e Eletroeletrônica de Ilhéus
CEP	Código de Endereçamento Postal
CNPJ	Cadastro Nacional da Pessoa Jurídica
CPF	Cadastro de Pessoa Física
CRUD	Criar, Ler, Atualizar e Deletar
DTO	Objeto de Transferência de Dados
HTTP	Protocolo de Transferência de Hipertexto
ORM	Objeto de Mapeamento Relacional
REST	Transferência de Estado Representacional
SGBD	Sistema de Gerenciamento de Banco de Dados
SOAP	Protocolo Simples de Acesso a Objetos
SQL	Linguagem de Consulta Estruturada
W3C	World Wide Web Consortium
WSDL	Web Services Description Language

SUMÁRIO

1 INTRODUÇÃO..... 4
1.1 Justificativa..... 5
1.2 Problema..... 5
1.3 Objetivos..... 6
1.3.1 Objetivo Geral..... 6
1.3.2 Objetivos Específicos..... 6
2 MATERIAIS E MÉTODOS..... 8
2.1 Padrão de projeto Adapter..... 8
2.2 Arquitetura de três camadas..... 8
2.3 Web Services..... 9
2.4 Teste de Software..... 10
2.5 .NET Plataform..... 12
2.6 PostgreSql..... 12
2.7 Visual Studio..... 12
2.8 DBeaver..... 13
2.9 Git..... 13
2.10 Entity Framework..... 14
2.11 Refit..... 15
2.12 AutoMapper..... 15
3 METODOLOGIA..... 17
3.1 Desenho Metodológico..... 17
3.1.1 Planejamento das Sprints..... 18
3.1.2 Execução das Sprints..... 18
3.1.3 Revisão e Ajustes..... 19
3.1.4 Testes e Implementação Piloto..... 19
4 DESENVOLVIMENTO..... 21
4.1 Análise de requisitos..... 21
4.1.1 Requisitos Funcionais..... 21
4.1.2 Requisitos Não Funcionais..... 23
4.2 Sobre o sistema..... 24
4.2.1 Arquitetura do sistema..... 24
4.2.2 Detalhamento dos módulos..... 24
4.2.4 Principais funcionalidades..... 27
4.2.5 Fluxos de Processos..... 28
4.2.6 Diagrama de sequência..... 29
5 RESULTADOS..... 31
5.1 Serviços de integração..... 31
5.1.1 Consulta de CNPJ..... 31
5.1.2 Consulta de CEP..... 32

5.1.3 Consulta de bancos.....33
5.2 Testes de software.....33
5.3 Impacto no desenvolvimento.....37
6 CONCLUSÃO..... 38
7 REFERÊNCIAS..... 39
APÊNDICE A - Tabela com o exemplo do serviço de obter por cnpj..... 41
APÊNDICE B - Swagger do método de obter por cnpj..... 43
APÊNDICE C - Json com a saída do método de obter por cnpj..... 43
APÊNDICE D - Tabela com o exemplo do serviço de obter por cnpj com parceiros..... 43
APÊNDICE E - Swagger do método de obter por cnpj com parceiros.....46
APÊNDICE F - Json com a saída do método de obter por cnpj com parceiros. 46
APÊNDICE G - Tabela com o exemplo do serviço de obter por cnpj com parceiros e seus cpfs.....47
APÊNDICE H - Swagger do método de obter por cnpj com parceiros e seus cpfs..... 49
APÊNDICE I - Json com a saída do método de obter por cnpj com parceiros e seus cpfs..... 50
APÊNDICE J - Tabela com o exemplo do serviço de obter pelo cep.....50
APÊNDICE K - Swagger do método de obter pelo cep.....51
APÊNDICE L - Json com a saída do método de obter pelo cep..... 51
APÊNDICE M - Tabela com o exemplo do serviço de obter estados.....52
APÊNDICE N - Swagger do método de obter estados.....53
APÊNDICE O - Json com a saída do método de obter estados.....53
APÊNDICE P - Tabela com o exemplo do serviço de obter todas cidades.....54
APÊNDICE Q - Swagger do método de obter todas cidades..... 55
APÊNDICE R - Json com saída do método de obter todas as cidades.....55
APÊNDICE S - Tabela com o exemplo do serviço de obter bancos por código.55
APÊNDICE T - Swagger do método de obter bancos por código.....56
APÊNDICE U - Json com a saída do método de obter bancos por código..... 56
APÊNDICE V - Tabela com o exemplo do serviço todos os bancos..... 56
APÊNDICE W - Swagger do método de obter todos os bancos..... 57
APÊNDICE X - Json com saída do método de obter todos os bancos..... 57

1 INTRODUÇÃO

Com o desenvolvimento da tecnologia e o crescimento dos sistemas nas mais diversas áreas, o desafio para integrar esses sistemas aumentou exponencialmente (FELTRIN, 2023). Dentro desse contexto, as APIs (Application Programming Interface), que são mecanismos que permitem a troca de informações entre diferentes sistemas (FELTRIN, 2023), surgem como uma solução essencial para facilitar essa integração e tornaram-se parte integrante do moderno ecossistema de software (MANIKAS, 2016). Ela fornece às empresas e organizações a capacidade de interconectar aplicativos, serviços e plataformas e explorar a capacidade de expandir valor de maneira mais eficiente e escalável (ABIGEE, 2016; ANUFF, 2017).

Historicamente, as APIs foram desenvolvidas visando permitir a comunicação entre softwares desde o surgimento dos computadores pessoais (IBM, 2016). No entanto, com o aparecimento das APIs na web por volta do ano 2000, observou-se um crescimento notável no interesse e uso dessas interfaces por programadores e pesquisadores. Tal popularidade deveu-se, em grande parte, à capacidade das APIs conectarem diversos sistemas e fomentarem a "economia das APIs", um cenário no qual essas interfaces são vistas como o tecido que mantém unida a complexa rede digital global (OFOEDA; BOATENG; EFFAH, 2019). Setores como o mercado de aplicativos móveis, são altamente dependentes de APIs para fornecer softwares confiáveis e interoperáveis, exemplificam como essas interfaces tornaram-se essenciais à inovação e sucesso empresarial (LINARES-VÁSQUEZ et al., 2013; BAVOTA et al., 2015).

De fato, as APIs desempenham um papel decisivo ao fornecerem um meio normatizado e eficiente para a integração entre sistemas diversos, facilitando a comunicação e a troca de informações entre os mesmos. Neste contexto, este projeto descreve o desenvolvimento e a implementação do Integration Hub, uma plataforma desenvolvida pelo Centro de Pesquisa e Desenvolvimento Tecnológico em Informática e Eletroeletrônica de Ilhéus (CEPEDI). O Integration Hub foi projetado para simplificar o processo de integração de múltiplas APIs e durante o desenvolvimento do projeto, o foco principal esteve na implementação e

configuração dos adaptadores necessários, garantindo uma comunicação eficiente e robusta entre o Integration Hub e os serviços externos.

1.1 Justificativa

Nos últimos anos, especialmente após 2006, o formato de comunicação digital predominante desenvolveu-se rapidamente devido ao aumento da largura de banda e à explosão da mídia social. Assim, agora na atual situação as APIs emergem como partes-chave cruciais não apenas para conectar sistemas entre si mas também em termos de inovação e ampliação de funcionalidade (DOERRFELD et al., 2016). Deve-se dizer que, no contexto atual, as APIs são uma solução central: elas permitem que sistemas diferentes se comuniquem e compartilhem dados de maneira eficiente, facilitando a interoperabilidade e conectividade entre plataformas variadas (FELTRIN, 2023).

O mercado de aplicativos para celulares, por exemplo, é um dos que mais apreciam esta tecnologia, pois usam as APIs para garantir a interoperabilidade e confiabilidade de seus sistemas, isso por sua vez permite inovações constantes (Linares-Vásquez et al., 2013; Bavota et al., 2015). Além disso, as APIs são vistas como a fibra que mantém unido o ecossistema digital, colocando a interconectividade nunca antes vista entre pessoas, aplicações e sistemas (OFOEDA; BOATENG; EFFAH, 2019).

Desta forma, este trabalho se justifica pelo fato de atender a uma necessidade emergente do mercado de tecnologia: a criação de um ambiente seguro e simplificado para a integração de métodos utilizando APIs. Reduzindo os esforços técnicos para a integração de muitas APIs, contribuindo de um modo geral para proporcionar um ambiente de desenvolvimento mais ágil, seguro e escalável, promovendo inovação e competitividade na área do desenvolvimento de software.

1.2 Problema

Diversas empresas enfrentam o problema de integração com várias APIs pela falta de padronização das mesmas, além da complexidade inerente dos sistemas.

Com múltiplas APIs que não são padronizadas e que frequentemente não se comunicam umas com as outras de maneira eficiente, elas não conseguem orquestrar a comunicação entre diferentes plataformas e serviços. Essa descentralização, por sua vez, leva à necessidade de processos de unificação manuais, manutenção cara e uma quantidade excessiva de riscos de segurança, compactuando dificuldades no tempo de resposta e na escalabilidade. Como resultado, elas não conseguem atender às demandas da empresa que exigem velocidade e precisão.

1.3 Objetivos

1.3.1 Objetivo Geral

Implementar e relatar o desenvolvimento de uma interface centralizada, denominada Integration Hub, com foco na implementação do módulo Adapters, que foi a contribuição principal do autor para o projeto. Esse módulo é responsável por abstrair e concentrar a lógica de integração com APIs externas, como busca de CEP, CNPJ e bancos. O objetivo geral é facilitar o acesso às funcionalidades dessas APIs por outros sistemas da empresa CEPEDI, sem que os desenvolvedores precisem se preocupar com a implementação e a manutenção dessas integrações.

1.3.2 Objetivos Específicos

Os seguintes objetivos específicos foram propostos para o desenvolvimento deste trabalho

a) Levantar os requisitos para a integração com APIs externas, a partir das demandas internas da empresa.
b) Aplicar a metodologia Scrum na condução do projeto, descrevendo as fases, sprints, entregas intercaladas e como essas etapas contribuíram para o progresso do desenvolvimento.
c) Projetar e implementar o Integration Hub, detalhando as etapas do processo de desenvolvimento, desde a definição da arquitetura até a escolha das tecnologias e práticas utilizadas.

d) Realizar testes no Integration Hub, avaliando o impacto no desempenho e na usabilidade do sistema.

2 MATERIAIS E MÉTODOS

Neste capítulo será feita a exposição das tecnologias e padrões fundamentais utilizados no desenvolvimento do sistema.

2.1 Padrão de projeto Adapter

O padrão Adapter é usado para converter uma interface de classe em outra interface necessária pelo cliente, isso é feito para permitir que classes com diferentes interfaces trabalhem juntas (GAMMA et al., 2000). O Adapter é igualmente chamado de wrapper e é usado para tornar utilizáveis classes em um contexto diferente, onde a interface não coincide (GAMMA et al., 2000).

No projeto, existe um módulo apenas para este padrão, responsável por centralizar toda a lógica de integração com os sistemas externos. No sistema, este padrão é utilizado para converter a interface específica que contém os endpoints da API externa para uma interface do próprio sistema, assim, abstraindo as informações da interface específica e facilitando a troca dessas APIs externas caso necessário.

2.2 Arquitetura de três camadas

A arquitetura de três camadas é um modelo aprimorado baseado no modelo de duas camadas que surgiu com o crescimento da Internet e a necessidade de combinar regras de negócio e fornecer escalabilidade e manutenção (BATTIST, 2003). Ela é aplicada em sistemas corporativos, especificamente em aplicativos Web, e tem sido identificada como um padrão para o desenvolvimento de sistemas robustos e escaláveis (MACORATTI, 2002b).

O cliente acessa o banco de dados de acordo com suas regras que são definidas no servidor de aplicação (BATTIST, 2003). O cliente não tem permissão para acessar o banco de dados diretamente, toda solicitação deve passar primeiro pelo servidor de aplicação . Assim, o sistema é dividido em três camadas:

- A camada de aplicação refere-se à forma como a interação entre o usuário e o software é gerenciada, sendo a principal função dessa camada mostrar informações para o usuário e converter suas ações em comandos que afetam o domínio e a camada de dados (FOWLER, 2007).
- Já a camada de dados é responsável pela comunicação com outros sistemas que realizam tarefas em benefício da aplicação, o principal e mais comum componente dessa lógica é o banco de dados, que armazena dados de forma persistente (FOWLER, 2007).
- A camada de negócios envolve as operações que a aplicação realiza dentro de seu campo específico, lidando com cálculos baseados nas entradas do usuário e nos dados armazenados, valida as informações que chegam da camada de apresentação e entende como aplicar as regras de negócio corretamente (FOWLER, 2007).

Essa é a arquitetura principal do projeto, sendo que a camada de aplicação que se trata da interação do usuário com o sistema, é representada pela solução IntegrationHub.API. A camada de negócios, que é responsável por tratar e validar todas as regras de negócio, é retratada pela solução IntegrationHub.Business. Por fim, a camada de dados que fica responsável pela persistência dos mesmos é representada pela solução IntegrationHub.Infraestructure.

2.3 Web Services

Web Services são aplicações publicadas, localizadas e invocadas através da Web. Elas podem servir para simples processos, como troca de mensagens, até complexas transações comerciais ou industriais. O acesso a um web service é feito via outras aplicações, podendo ser outros web services, para a obtenção de dados ou interação com serviços (TAMAE; LIMA, 2005).

A W3C (consórcio de empresas de tecnologia para definição de padrões para a Web) definiu um padrão de interface para possibilitar a interação com web services, o WSDL (Web Services Description Languages), que funciona como um índice dos métodos oferecidos. A comunicação ocorre via protocolo como SOAP e REST (VIGO et al., 2014).

O protocolo Rest (*Representational State Transfer*) tem como principais características (MORO; DORNELES; REBONATTO, 2011) [Fielding 2000]:

- Tramitação de dados utilizando protocolo HTTP, utilizando seus métodos, como GET, POST, PUT e DELETE, para indicar a ação que se deseja realizar.
- Característica utilizada é a cliente-servidor, onde um servidor disponibiliza serviços que o cliente, que pode ser outro servidor, pode acessar.
- Diferente do protocolo SOAP não possui estado no servidor, desta forma a cada requisição é necessário passar todas as informações necessárias.

O Integration Hub usa Web Services para se comunicar com APIs externas, como serviços de busca de CEP e de CNPJ.

2.4 Teste de Software

Teste de software é o processo de execução da aplicação de forma a determinar se as especificações foram atingidas e se houve o correto funcionamento no ambiente para o qual ele foi projetado, de forma a identificar falhas em um produto para que possa ocorrer a correção antes da entrega (SILVA; SILVA; CAROSIA, 2021).

(DA SILVA et al., 2016) separa as técnicas de testes em dois grupos: estrutural e funcional. Eles são descritos como:

- Estrutural: Testa o código fonte, testa os fluxos básicos e alternativos, entre outros, pode ser sumarizado como:
    - Teste de estresse: Verifica como o sistema é executado com determinados volumes de dados.
    - Teste de segurança: Verifica se o sistema é protegido de acordo com normas e políticas da organização.
    - Teste de execução: Verifica se o sistema atingiu um nível desejado de eficiência

    - Teste de contingência: Avalia se o sistema é capaz de retornar ao nível anterior à falha.
    - Teste de operação: Verifica-se que o sistema opera de acordo com sua documentação.
    - Teste de conformidade: Avalia-se se o sistema foi desenvolvido de acordo com padrões.

- Funcional: Tem como objetivo testar se os requisitos estão funcionando conforme o especificado, isto é, faz uma simulação de inserção de dados e verifica a saída, sem preocupação com o que está por trás. Pode se dividir em:
    - Teste de requisitos: Se o sistema foi ou não efetuado conforme suas especificações.
    - Teste de regressão: O sistema ou algumas de suas partes foram afetados pela mudança.
    - Teste de tratamento de erros: Quando um software prevê que um determinado problema pode ocorrer e prepara um plano para este antes de acontecer a falha.
    - Teste de suporte manual: Interação entre homem e máquina.
    - Teste de interconexão com outros softwares: Se o sistema aceita as informações de outros ou envia estas informações em uma linguagem comum.
    - Teste de controle: Verificar se o sistema possui alguma governança de dados, validações e integridade, logs de auditoria;
    - Testes paralelos: A versão antiga gera os mesmos resultados quanto a versão atual.

Esses testes foram executados em todas as camadas do sistema, com atenção especial às funcionalidades críticas, como a comunicação com APIs externas e a persistência dos dados.

2.5 *.NET Plataform*

É uma plataforma desenvolvida pela Microsoft para o desenvolvimento e execução de sistemas e aplicações. São suportadas várias linguagens de programação, como C# e F#, e os sistemas desenvolvidos podem rodar sobre qualquer sistema operacional que possuam o framework instalado. Nele podem ser desenvolvidas aplicações desktop, web e mobile.

O Integration Hub foi desenvolvido nessa plataforma, utilizando a linguagem C# e algumas das tecnologias disponibilizadas por esse ecossistema.

2.6 PostgreSql

PostgreSQL é um sistema de gerenciamento de banco de dados relacional de código aberto conhecido por sua robustez, flexibilidade e conformidade padrão com SQL. Ele suporta uma ampla variedade de tipos de dados, incluindo tipos complexos e extensíveis, e permite realizar operações avançadas, como executar consultas em JSON e interoperar com linguagens de programação como Python e PL/pgSQL para criar funções como procedimentos armazenados. PostgreSQL é o banco de dados primário selecionado para este projeto devido ao seu alto desempenho, confiabilidade e escalabilidade. Além disso, o suporte avançado à integridade referencial e de transações garante a consistência dos dados, e os amplos recursos de extensibilidade permitem a personalização para atender aos requisitos específicos do projeto. Uma comunidade ativa e uma vasta gama de ferramentas e extensões disponíveis tornam o PostgreSQL uma solução de gerenciamento de dados versátil e poderosa para ambientes de produção.

O PostgreSQL foi o sistema de gerenciamento de banco de dados escolhido para o Integration Hub devido à sua robustez e escalabilidade. Ele é utilizado pela camada de dados (IntegrationHub.Infrastructure) para gerenciar a persistência das informações.

2.7 Visual Studio

Software desenvolvido pela Microsoft que atua como um ambiente de desenvolvimento integrado, especialmente para o .NET framework e outras linguagens, como Visual Basic, C# e F#. Ele possibilita não só o desenvolvimento de aplicações web, mas também de aplicações desktop, mobile e para a nuvem.

Para o desenvolvimento de uma aplicação web com o.NET framework o Visual Studio oferece inúmeros benefícios como a fácil execução do projeto, a possibilidade de edição durante tal execução, o IntelliSense, que atua como se fosse um auto-complete poderoso, refatoração do código, assistente de exceção e muitas outras.

O Visual Studio foi o ambiente de desenvolvimento integrado (IDE) utilizado para construir o Integration Hub. Ele permitiu uma experiência eficiente para os desenvolvedores, com ferramentas como IntelliSense, refatoração e depuração.

2.8 DBeaver

DBeaver é um software livre e uma ferramenta de administração e gerenciamento de bancos de dados comumente utilizado entre desenvolvedores e administradores de banco de dados. Ele suporta vários SGBDs como PostgreSQL, MySQL e SqlServer. As funcionalidades do DBeaver podem ser: executar comandos SQL, navegar e gerenciar bases de dados, editar dados, criar, modificar e excluir objetos do banco e muito mais. Existem algumas características no software que podem ser utilizadas para suas diversas funcionalidades; ele oferece funcionalidades de SQL e quase todas as funcionalidades de consulta, diagramas ER e muito mais.

O DBeaver foi utilizado como ferramenta de administração e gerenciamento do banco de dados PostgreSQL. Durante o desenvolvimento do Integration Hub, ele auxiliou na execução de comandos SQL e visualização das tabelas.

2.9 Git

O Git é um sistema de controle de versão distribuído de código aberto gratuito

que se tornou bastante popular para o desenvolvimento de software. O Git é capaz de manter projetos de qualquer tamanho e praticamente não reduzir a velocidade durante a expansão do projeto. Isso torna o sistema extremamente eficiente. O Git possui várias funcionalidades básicas: commit e branch são duas das mais usadas, que, de um lado, permitem que você salve as mudanças e, do outro, crie um novo ramo.

O Git foi empregado para controle de versão e gerenciamento do código-fonte do Integration Hub. Através de branches específicas para cada tarefa, os desenvolvedores puderam trabalhar de maneira paralela e organizada, garantindo a rastreabilidade das alterações e a integração contínua do código.

2.10 Entity Framework

É um framework de Mapeamento Objeto-Relacional (ORM, do inglês Object-Relational-Mapping) para a plataforma .NET, que permite aos desenvolvedores interagirem com o banco de dados utilizando objetos .NET, sem a necessidade de escrever código SQL diretamente. Com isso, é possível trabalhar em um nível elevado de abstração ao manipular os dados.

Além de dispensar a preocupação do programador com a escrita de SQL, o framework também abstrai o banco de dados utilizado. Dessa forma, os dados podem ser armazenados em diferentes sistemas, cada um com seu próprio protocolo, e o Entity gerencia a conexão e interação. Caso haja a necessidade de trocar o SGBD (Sistema de Gerenciamento de Banco de Dados), basta ajustar as configurações do Entity, sem que o programador precise modificar o código de sua aplicação.

O Entity Framework foi utilizado na camada de persistência de dados (IntegrationHub.Infrastructure). Ele foi responsável por gerenciar as operações CRUD (Create, Read, Update, Delete) e manter a consistência dos dados no banco PostgreSQL, permitindo a comunicação fluida entre as camadas de negócios e o banco de dados.

2.11 Refit

É uma biblioteca .NET que facilita a integração com APIs REST, permitindo a criação de um cliente HTTP através de interfaces e atributos que descrevem as chamadas. Ele permite que as definições de API sejam descritas como interfaces simples no código e faz com que o Refit gere as implementações necessárias para realizar as solicitações HTTP. Dessa forma, gera um código muito mais legível e de fácil manutenção. A escolha do Refit para o desenvolvimento do IntegrationHub foi feita devido à sua capacidade de abstrair a complexidade das solicitações HTTP, permitindo ao desenvolvedor focar muito mais nas regras de negócio ao invés de se preocupar com os detalhes de implementação dessas chamadas externas de API.

O Refit foi utilizado na camada dos adaptadores (IntegrationHub.Adapters). Ele foi utilizado para integrar o sistema com APIs externas, como as APIs de consulta de CEP, CNPJ e bancos, garantindo que essas integrações fossem realizadas de maneira eficiente e com menos código manual.

2.12 AutoMapper

É um mapeador de objetos .NET que permite converter um tipo de objeto automaticamente em um outro tipo. Durante o desenvolvimento do IntegrationHub, foi empregado o AutoMapper para simplificar o mapeamento entre DTOs e os modelos das entidades, o que eliminou a necessidade de escrita do código de mapeamento e consequentemente, tornou o código mais limpo, legível e consistente. Portanto, o AutoMapper facilita a transformação de dados entre as camadas do sistema, especialmente ao disponibilizá-los via API, assim as transformações ficam automatizadas e não manipuladas com menos propensão a erros de programação e permitem manter as classes de apresentação e persistência separadas.

O AutoMapper foi empregado na camada de negócios (IntegrationHub.Business) e na camada de API (IntegrationHub.Api). Ele foi utilizado para mapear dados entre as camadas de apresentação e persistência, facilitando a comunicação entre os controladores, os serviços e o banco de dados, garantindo

que os dados fossem transformados corretamente ao trafegar entre os diferentes níveis do sistema.

3 METODOLOGIA

Neste capítulo será apresentada a metodologia que foi utilizada no desenvolvimento do sistema. Será demonstrado o desenho metodológico da aplicação, com todas as fases planejadas e realizadas para a conclusão do projeto.

3.1 Desenho Metodológico

O projeto do IntegrationHub foi realizado através da metodologia ágil conhecida como Scrum, modelo adotado pelo CEPEDI, a fim de obter o máximo de eficiência e flexibilidade em projetos desenvolvidos internamente. O IntegrationHub, entretanto, não foi desenvolvido para um cliente em específico, na verdade trata-se de uma solução desenvolvida internamente pela empresa para, assim, diminuir as dificuldades em relação à integração com APIs desenvolvida externamente em diferentes projetos executados para clientes diversos.

O desenvolvimento do projeto foi dividido em quatro fases principais: Planejamento das Sprints, Execução das Sprints, Revisão e Ajustes e Implementação Piloto. Essas etapas foram devidamente organizadas, para garantir adequação contínua da tecnologia às suas necessidades internas. A Figura 1 ilustra todo esse fluxo:

Figura 1 - Descrição do fluxo da metodologia scrum



Figura adaptada da fonte:
SOMMERVILLE, 2007

A seguir, serão detalhadas todas as etapas envolvidas nessa metodologia adotada.

3.1.1 Planejamento das Sprints

O desenvolvimento deste projeto se originou na identificação das necessidades internas da empresa. Sob a liderança de um Scrum Master, a equipe trabalhou em conjunto para criar uma plataforma sólida que simplifica a integração de vários sistemas com APIs externas, como as de CEP ou consulta de CNPJ. O backlog do produto foi elaborado de forma colaborativa entre os membros da equipe e o Product Owner, levando em consideração as demandas frequentes de integração enfrentadas pelos desenvolvedores e gerentes de projetos da empresa.

As funcionalidades essenciais foram priorizadas no backlog e organizadas em sprints curtos de duas semanas, sob a supervisão do Product Owner. Esse planejamento permitiu à equipe entregar valor rapidamente, garantindo coesão e flexibilidade para ajustar as tarefas às necessidades internas identificadas durante as revisões.

3.1.2 Execução das Sprints

A execução das sprints foi conduzida em um ambiente de colaboração contínua entre os membros da equipe, com o Scrum Master atuando como facilitador para garantir que o progresso fosse mantido e os impedimentos fossem rapidamente resolvidos. Durante as reuniões diárias de feedback, o Scrum Master liderava as discussões, garantindo alinhamento entre as tarefas de cada membro e os objetivos gerais do projeto.

As principais atividades desenvolvidas ao longo das sprints incluíram:

- Sprint 1: Levantamento das necessidades internas da empresa e definição de requisitos.

- Sprint 2: Configuração inicial do projeto e integração com a API de CEP.
- Sprint 3: Expansão da funcionalidade para integrar API de CNPJ e também API de consulta de bancos.
- Sprint 4: Realização de testes de usabilidade com desenvolvedores internos e ajustes de funcionalidade.

3.1.3 Revisão e Ajustes

Ao término de cada ciclo, foram realizadas reuniões de retrospectiva conduzidas pelo Scrum Master, onde a equipe avaliava as funcionalidades desenvolvidas e discutia melhorias para os próximos ciclos. Essas reuniões incluíram os principais interessados internos, como desenvolvedores e gestores de projetos, que contribuíram com feedback para refinar o desenvolvimento da plataforma.

Posteriormente, o Scrum Master liderava as reuniões de planejamento das sprints seguintes, organizando as tarefas com base nos feedbacks recebidos e nas prioridades do backlog. Essa abordagem garantiu que todos os membros da equipe estivessem alinhados e que o produto final atendesse às necessidades práticas e técnicas dos projetos da empresa.

3.1.4 Testes e Implementação Piloto

Depois de terminadas as sprints de desenvolvimento, o projeto entra na fase de implementação piloto e testes finais. Nesta fase, foi estabelecido um ambiente controlado onde se utilizou o IntegrationHub num aplicativo administrativo mas ainda interno em sua aplicação real para verificar como seria essa integração em termos de facilidades com APIs externas.

As funções, segurança, desempenho e facilidade de uso foram todos experimentados em campo. Foi então permitido que programadores, que já eram usuários das plataformas de desenvolvimento internas, trabalhassem com as informações. Só então veremos onde há um programa insatisfatório. Foram necessários fazer adaptações, uma de cada vez, para garantirmos que o produto

final mantivesse todos os elementos de usabilidade e desempenho que ele deveria ter.

4 DESENVOLVIMENTO

O desenvolvimento do Integration Hub contou com a participação de uma equipe composta por 12 profissionais, incluindo desenvolvedores, um líder, desenvolvedores de operações e um testador. O foco principal esteve na implementação e configuração do módulo de Adapters, responsável por realizar a integração com APIs externas, como serviços de CEP, CNPJ e bancos.

O desenvolvimento foi iniciado a partir do levantamento e análise de requisitos. Com esta parte concluída, foi possível construir os diagramas e modelos necessários para o desenvolvimento da aplicação, como também a definição das ferramentas e tecnologias a serem utilizadas na implementação dos requisitos levantados.

4.1 Análise de requisitos

Os requisitos representam, de forma direta e resumida, as funcionalidades e características do sistema. Eles são extraídos após o processo de análise das necessidades do cliente e através da coleta de dados.

Os próximos tópicos apresentarão os requisitos funcionais e não funcionais que delimitam o sistema de acordo com os objetivos citados por (MAGALHÃES; PINHEIRO, 2007).

4.1.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funções que o sistema deve realizar. Eles variam de acordo com o tipo do software que está sendo desenvolvido, sendo afetado pelo tipo do sistema e o usuário no qual ele se destina (SOMMERVILLE, 2007).

O correto e bom detalhamento dos requisitos é extremamente importante para se obter as funcionalidades que se deseja no sistema.

A tabela a seguir lista o número, nome e classificação dos requisitos. Eles foram classificados em essencial (E), importante (I) e desejável (D). Logo após é realizada uma descrição mais detalhada de cada requisito.

Tabela 1 - Requisitos Funcionais

Requisito	Nome	Classificação
RF1	Conexão com APIs Externas	Essencial
RF2	Monitoramento de Status das APIs	Desejável
RF3	Armazenamento de logs	Desejável
RF4	Gestão de Permissões de Usuário	Importante
RF5	Integração com API de CEP	Essencial
RF6	Integração com API de CPF	Essencial
RF7	Integração com API de CNPJ	Essencial
RF8	Integração com API de Bancos	Essencial
RF9	Envio de Emails	Desejável
RF10	Gerenciamento de arquivos	Desejável

RF1: O sistema precisa ser capaz de se conectar a APIs externas, garantindo a segurança e facilidade no compartilhamento de informações.

RF2: O sistema deve observar o status das APIs conectadas, certificando-se de que elas estejam disponíveis.

RF3: O sistema deve armazenar um log de todas as operações de criação e alteração de dados. Esse log é usado para fins de auditoria, depuração e melhoria de desempenho.

RF4: O sistema deve habilitar um sistema de permissões, garantindo que apenas usuários autorizados possam ver informações.

RF5: O sistema deve se conectar a uma API de consulta de CEP e permitir a busca do endereço com base no CEP, e concordar com APIs de cidades e estados.

RF6: O sistema deve poder consultar uma API de CPF.

RF7: O sistema deve poder consultar outra API de CNPJ.

RF8: O sistema deve se conectar a uma API bancária para ser capaz de saber quais bancos estão atualmente em operação.

RF9: O sistema deve ser capaz de enviar emails.

RF10: O sistema deve ser capaz de administrar arquivos, tornando possível enviar e salvar arquivos em outros programas, como o Google Drive.

4.1.2 Requisitos Não Funcionais

Conforme (SOMMERVILLE, 2007), os requisitos não funcionais não estão diretamente associados às funcionalidades específicas do sistema, mas sim às características do mesmo, como confiabilidade, tempo de resposta e capacidade de armazenamento.

O número, nome e classificação dos requisitos estão listados na tabela abaixo. Eles foram classificados em essencial (E), importante (I) e desejável (D). Logo após é realizada uma descrição mais detalhada de cada requisito.

Tabela 2 - Requisitos não funcionais

Requisito	Nome	Classificação
RNF1	Utilizar linguagem C# e .NET framework.	Essencial
RNF2	Utilizar o banco de dados PostgreSql	Essencial
RNF3	Hospedagem em um servidor da Azure	Essencial
RNF4	Tolerância a falhas	Importante

RNF1: A linguagem C# e o .NET framework devem ser utilizados para o desenvolvimento do sistema.

RNF2: A persistência dos dados deve ser feita utilizando o banco de dados PostgreSql.

RNF3: O sistema deve ser hospedado em um servidor na nuvem da Azure.

RNF4: O sistema deve detectar e gerenciar falhas de conexão com APIs externas.

4.2 Sobre o sistema

O Integration Hub foi projetado para atuar como um ponto central de integração entre os diversos sistemas e APIs utilizadas pela empresa. Além de simplificar a conexão entre múltiplas interfaces, ele melhora a eficiência operacional, reduzindo a necessidade de adaptações manuais e otimizando a manutenção das integrações existentes. Esses recursos permitem à empresa concentrar seus esforços no desenvolvimento de funcionalidades de negócio, garantindo que as integrações ocorram de forma transparente e robusta.

4.2.1 Arquitetura do sistema

A arquitetura do Integration Hub foi projetada utilizando o padrão de design em três camadas, com forte ênfase na separação das responsabilidades. As três camadas são:

- Camada de apresentação: Responsável pela interação com o usuário, onde estão localizados os pontos de extremidade que recebem as solicitações.
- Camada de negócios: Gerencia as regras do negócio, contendo toda a lógica do sistema.
- Camada de persistência de dados: Onde é realizada a persistência no banco do Postgresql.

4.2.2 Detalhamento dos módulos

- Business: Este módulo é responsável por centralizar a lógica de negócio da aplicação, definindo, portanto, as regras a serem seguidas pelo IntegrationHub. As suas principais subpastas e as suas funções estão descritas abaixo:
    - DTOs: Armazena classes de objetos de transferência de dados, assim definidas para carregar dados entre as camadas do sistema sem expor diretamente entidades do banco de dados.

    - Enums: Armazena, a título de exemplo, enumeradores, que servem para definir diversos valores fixos, como conjunto de status visando a padronização do código.
    - Extensions: Onde ficam os extensions methods do C#.
    - Interfaces: Contém as interfaces referentes às classes de negócio. As interfaces garantem que suas implementações sigam contratos, o que facilita alterações futuras ou mesmo uma implementação completamente nova caso seja necessário.
    - Models: Armazena a modelagem de dados usada nas entidades do sistema, que são mapeadas para o banco de dados.
    - Notifications: Armazena variáveis de notificação e é responsável por manter a organização e ordem da comunicação de eventos.
    - Services: Armazena as classes de serviço que implementam essas regras e, portanto; efetuam a manipulação das operações e interações principais do sistema.

- Infrastructure: Esse módulo lida com a infraestrutura e a persistência de dados, conectando a lógica de negócios ao banco de dados e gerenciando a comunicação com o sistema de armazenamento. O módulo consiste dos submódulos:
    - Data: Gerencia o acesso aos dados; incluindo as configurações de conexão com o banco de dados e classes de contexto do Entity Framework.
    - Migrations: Mantém as migrações do banco de dados. Migrations ajuda a manter o esquema atualizado à medida que o desenvolvimento avança.

- Api: Este é o módulo de interface do usuário ou outros sistemas; é responsável por expor os endpoints da API. Ele contém as subpastas a seguir.
    - Controllers: Contém os controladores da API que expõem os endpoints. Cada controlador é responsável por responder a um grupo

de operações específico, e eles são responsáveis por receber requisições HTTP, processá-las e retornar respostas apropriadas.

- AutoMappers: Contém os arquivos de mapeamento, é uma classe de configuração do AutoMapper. AutoMapper é uma biblioteca que é usada para converter modelos de exibição como viewModels em DTOs e DTOs em entidades e vice-versa, isso facilita a conversão de modelos de exibição de um modelo para outro, o uso de mecanismos diferentes e a conversão.
- InputModels: Define os modelos de entrada para solicitações. Estes são os modelos que o sistema espera receber de seus clientes externos ao consumir a API.
- ViewModel: Esses são os modelos de resposta da API para o cliente. São modelos de resposta que são retornados da API para o frontend, geralmente modelados para obter apenas o necessário.

- Adapters: Este módulo foi o foco principal durante o desenvolvimento do projeto. Ele é responsável por fazer a integração com APIs externas. Cada submódulo representa um adaptador específico para um serviço de API, encapsulando a complexidade da integração. O foco do projeto envolveu a implementação dos adaptadores, configuração das chamadas e tratamento das respostas, garantindo que o sistema pudesse se comunicar de forma padronizada e eficiente com serviços externos. Dentro da pasta Adapters existe uma solução para cada API externa conectada:
  - IntegrationHub.Adapters.BrasilAbertoApi
  - IntegrationHub.Adapters.BrasilApi
  - IntegrationHub.Adapters.Ibge
  - IntegrationHub.Adapters.Serpro

4.2.3 Detalhamento dos Adapters

A estrutura dos adaptadores é composta pelos seguintes diretórios e arquivos:

- AutoMappers: Contém arquivos responsáveis pelo mapeamento automático entre DTOs e modelos internos.
- DTOs: Define os objetos de transferência de dados, que representam os dados recebidos da API externa.
- HealthChecks: Monitora a disponibilidade e a integridade do serviço externo.
- Interfaces: Define contratos para implementação dos serviços de adaptação.

Classes Principais:

- AdapterService.cs: Implementa o serviço principal para comunicação com a API, contendo métodos como GetByZipCode, por exemplo.
- DependencyInjectionConfig.cs: Configura a injeção de dependências do adaptador.
- ApiConfig.cs: Contém configurações para clientes HTTP usando a biblioteca Refit e políticas de retry com Polly para garantir a resiliência em falhas de comunicação.
- ApiSettings.cs: Centraliza as configurações do adaptador, como URL base e parâmetros de retry.

Exemplo de fluxo:

1. Solicitação: O controlador recebe a solicitação do usuário e a encaminha ao serviço correspondente.
2. Chamada ao Adaptador: O serviço realiza a chamada ao Adapter correspondente, que por sua vez faz a requisição HTTP para a API externa.
3. Tratamento da Resposta: O adaptador processa a resposta e converte os dados recebidos em um modelo padronizado para o sistema.
4. Retorno ao Cliente: O controlador devolve a resposta ao cliente, formatada como ViewModel.

4.2.4 Principais funcionalidades

- Integração com APIs Externas: O sistema é capaz de configurar e administrar múltiplas APIs, atualmente inclui a API responsável por consultar CEP e

CNPJ e consulta de bancos. Cada uma é configurada separadamente, suas credenciais e pontos de extremidade (endpoints) são configurados, o que simplifica a abordagem para se comunicar com os serviços autônomos de terceiros.

- Facilidade de expansão: Por sua vez, o sistema foi desenvolvido com flexibilidade em mente, tornando possível adicionar novas APIs sem uma reengenharia extensa da arquitetura existente.

4.2.5 Fluxos de Processos

O fluxo de processos apresenta o movimento de uma solicitação por dentro do sistema, mostrando como a solicitação passa por diferentes camadas que colaboram para atendê-la. A seguir, veja um caso de fluxo comum, partindo da solicitação até o retorno do resultado final.

1) Solicitação inicial:
    a) Camada de controle: As solicitações HTTP são recebidas no controlador na camada IntegrationHub.Api. Os controladores validam se as informações recebidas estão de acordo com o esperado e passam a chamada para a camada de serviços.

2) Processamento da lógica de negócios:
    a) Chamada aos serviços: O controlador vai encaminhar a chamada para o serviço específico e esse por sua vez vai ser responsável por fazer as validações relacionadas a lógica de negócio e passar essa chamada para a camada de persistência ou então para a camada de Adapters, caso haja a necessidade.
    b) DTOs e Mapeamento: O serviço faz uso de DTOs ou modelos para manter as informações que vêm da camada de API e mapeiam para converter o objeto recebido conforme DTOs ou modelos.
    c) Validações e Notificações: A camada de negócios valida a solicitação de dados e a usa para enviar alertas do serviço e dados de erro a classe de Notificação.

3) Comunicação com APIs
    a) Chamada externa: Caso o fluxo exija a comunicação com APIs de terceiros, a chamada é enviada a um adaptador especializado na pasta IntegrationHub.Adapters.
    b) Tratamento de Resposta: O adaptador faz a chamada à API de terceiros e recebe a resposta propriamente dita ou uma mensagem de erro, convertendo-a em um tipo de dado padrão para camada do sistema.

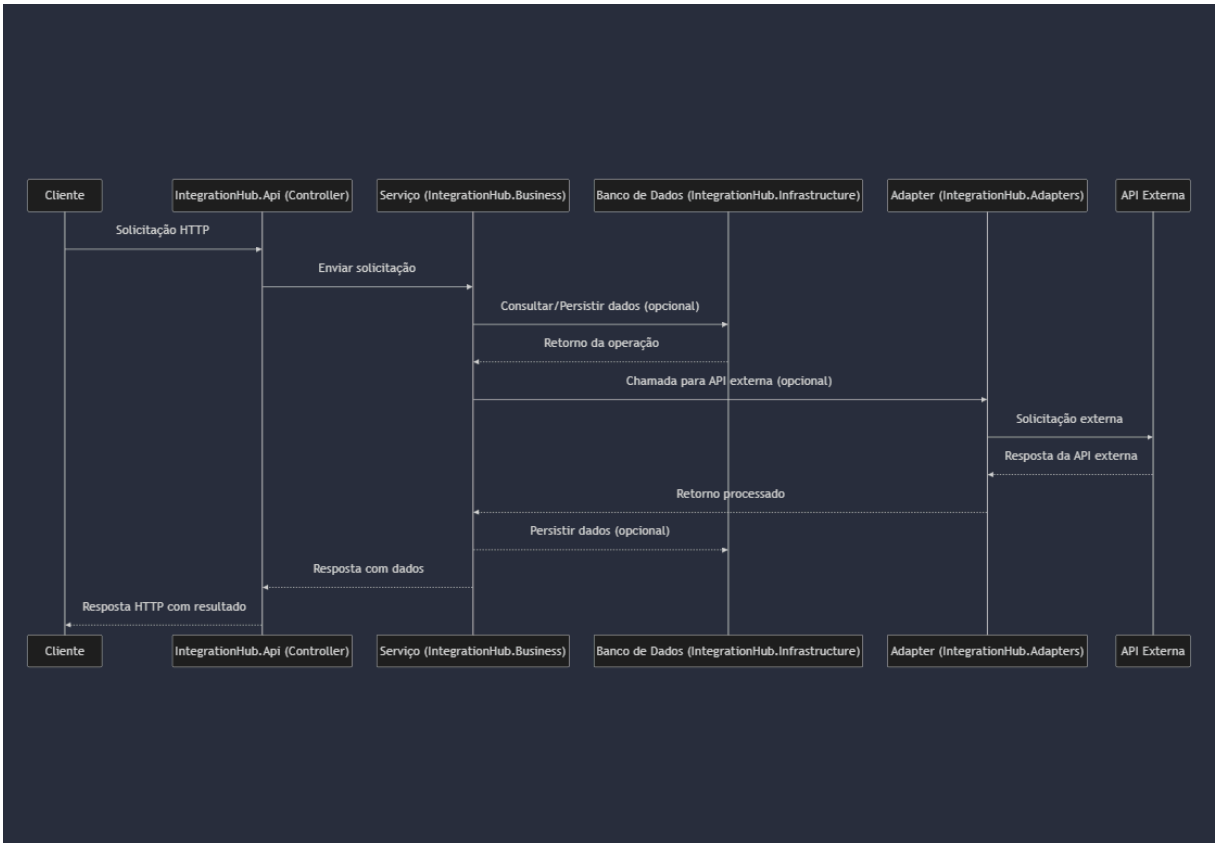
4) Armazenamento e Recebimento:
    a) Camada de persistência: A solicitação salva ou consulta informações no banco de dados com a camada Infrastructure. Os serviços disparam as operações no contexto do banco de dados, que usa uma ORM, o Entity Framework.

5) Resposta e Tempo:
    a) ViewModels e saída: Os serviços de negócios retornam o resultado para o controlador, que os transforma em ViewModel e retorna a mensagem final.
    b) Resposta: O controlador retorna uma resposta HTTP contendo os dados requisitados ou uma mensagem de erro, caso algum problema tenha sido detectado no fluxo.

4.2.6 Diagrama de sequência

Para uma melhor ilustração, segue abaixo o diagrama de sequência que ilustra o fluxo descrito no tópico anterior.

Figura 2 - Diagrama de sequência contendo o fluxo de uma requisição no sistema



5 RESULTADOS

Com a finalização da implementação dos módulos propostos, temos o sistema final pronto. Como a implementação ocorreu com sucesso, passamos à fase de verificação e validação do sistema para os requisitos levantados no início do projeto.

5.1 Serviços de integração

O sistema mantém comunicação com vários outros, que são os sistemas de consulta de CNPJ e CEP, por exemplo. A maioria das ações executadas é voltada para a importação de informações. Abaixo está a maneira como é feita e as informações recebidas em cada comunicação.

5.1.1 Consulta de CNPJ

A integração com a API de consulta de CNPJ permite obter informações da empresa com base no número do cadastro do CNPJ. A comunicação é feita por meio de requisições REST com o protocolo HTTPS e dados no formato JSON. Assim, o sistema pode obter informações essenciais sobre as empresas, tais como a denotação social, o endereço, a denominação corporativa, etc. Abaixo estão os serviços disponibilizados:

- GetByCnpj(cnpj)

  Parâmetros: Uma string qualquer.

  Ação: Retornar os dados de uma empresa a partir de um cnpj informado.

  Retorno: Dados de uma empresa.

  Exemplo: Apêndice A

- GetByCnpjWithPartners(cnpj)

  Parâmetros: Uma string qualquer.

  Ação: Retornar dados de uma empresa incluindo uma lista com os seus parceiros.

Retorno: Dados de uma empresa contendo a lista de parceiros.

Exemplo: Apêndice D

- GetByCnpjWithPartnersCpf(cnpj):

  Parâmetros: Uma string qualquer.

  Ação: Retornar dados de uma empresa contendo a lista de seus parceiros juntamente com os seus CPFs.

  Retorno: Dados de uma empresa com parceiros e os CPFs dos parceiros.

  Exemplo: Apêndice G

5.1.2 Consulta de CEP

Para obter dados de endereço, o sistema se comunica com o serviço de consulta de CEP, permitindo que o usuário obtenha rapidamente o endereço completo ao informar o CEP. Abaixo é detalhado a forma como foi feita e as informações recebidas em cada comunicação.

- GetByZipCode(zipCode)

  Parâmetros: Uma string qualquer.

  Ação: Retornar os detalhes de um endereço completo a partir do CEP, incluindo rua, bairro, cidade e estado.

  Retorno: Dados de um endereço.

  Exemplo: Apêndice J

- GetStates

  Ação: Retornar uma lista paginada com todos os estados do Brasil.

  Retorno: Lista de todos os estados do Brasil.

  Exemplo: Apêndice M

- GetAllCities

  Ação: Retornar uma lista com todas as cidades do Brasil.

  Retorno: Lista de todas as cidades do Brasil.

  Exemplo: Apêndice P

5.1.3 Consulta de bancos

A integração com o sistema de bancos retorna algumas informações sobre instituições financeiras. Abaixo é detalhado a forma como foi feita e as informações recebidas em cada comunicação.

- GetByCode(code)

  Ação: Consultar os detalhes básicos de um banco específico a partir de um código.

  Retorno: Lista contendo os detalhes básicos de um banco.

  Exemplo: Apêndice S

- GetAll

  Ação: Consultar todos os bancos registrados.

  Retorno: Lista de todos os bancos registrados.

  Exemplo: Apêndice V

5.2 Testes de software

SOMMERVILLE (2007) apresenta dois tipos de teste de software: testes de caixa branca e testes de caixa preta. O primeiro diz respeito ao software produzido, analisando principalmente o código da implementação. Para isso, são feitas entradas no projeto para verificar como foi feito, qual a lógica utilizada e etc. Testes de caixa preta, por sua vez, não se importam com a forma, as técnicas e etc, analisando apenas o comportamento do software de acordo com as entradas fornecidas e as saídas recebidas.

Testes de caixa branca eram realizados após a implementação de uma nova funcionalidade para uma procura de problemas relacionados à lógica de programação, que eram corrigidos o mais rápido possível. Os testes de caixa preta, por sua vez, foram realizados após o desenvolvimento do Banco de Dados e de cada módulo.

Os testes foram realizados em dois ambientes: um ambiente controlado, que era pelo próprio desenvolvedor e um ambiente real, onde o usuário poderia perceber se estava satisfeito. As tabelas a seguir descrevem os testes feitos, que abrangeram o banco de dados e as integrações com sistemas externos.

Tabela 1 - Descrição do teste de integração com o serviço de consulta de CNPJ

CASO DE TESTE	
Código	1 - Teste de integração com os serviço de consulta de CNPJ
Finalidade	Verificar se o sistema recupera corretamente as informações de empresas com base no CNPJ informado
Entradas	Consulta com diferentes números de CNPJ válidos e inválidos
Resultados esperados	Retorno correto de dados para CNPJs válidos; mensagem de erro para CNPJs inválidos
Dependências	Serviço de consulta de CNPJ ativo e conexão de rede
PROCEDIMENTOS	
Preparação	Verificar conectividade com o serviço de CNPJ
Inicialização	Configurar parâmetros de API
Execução	Realizar consultas de CNPJ e validar retornos
Recursos específicos	Ambiente de testes com API ativa
AVALIAÇÃO	
Resultados obtidos	Os resultados esperados foram alcançados, confirmando a integridade da integração.

Tabela 2 - Descrição do teste de integração com o serviço de consulta de CEP

CASO DE TESTE	
Código	2 - Teste de integração com os serviço de consulta de CEP
Finalidade	Validar se o sistema consegue retornar informações de endereços com base nos CEPs informados
Entradas	Consulta com CEPs válidos e inválidos
Resultados esperados	Retorno de endereço completo para CEPs válidos e mensagem de erro para CEPs inválidos
Dependências	Serviço de consulta de CEP ativo
PROCEDIMENTOS	
Preparação	Configurar acesso à API de CEP
Inicialização	Definir CEPs para teste
Execução	Realizar consultas e verificar precisão das informações
Recursos específicos	Ferramentas de integração e rede disponível
AVALIAÇÃO	
Resultados obtidos	Os resultados esperados foram alcançados, garantindo a funcionalidade da consulta de CEP.

Tabela 3 - Descrição do teste de integração com a API de bancos

CASO DE TESTE	
Código	3 - Teste de Integração com a API de Bancos
Finalidade	Verificar se o sistema consegue consultar e validar informações bancárias corretamente
Entradas	Consultas de dados bancários válidos e inválidos
Resultados esperados	Retorno correto para dados bancários válidos; mensagens de erro para entradas inválidas

Dependências	API de bancos configurada e acessível
PROCEDIMENTOS	
Preparação	Configurar os parâmetros de acesso à API de bancos
Inicialização	Definir dados bancários para teste
Execução	Realizar consultas e validar os retornos
Recursos específicos	Ferramentas de teste para integração com a API
AVALIAÇÃO	
Resultados obtidos	Os dados foram retornados de forma correta, cumprindo os requisitos esperados.

Tabela 4 - Descrição do teste de banco de dados

CASO DE TESTE	
Código	4 - Teste consistência do banco de dados
Finalidade	Verificar a consistência e integridade das operações no banco de dados
Entradas	Operações de inserção, atualização, leitura e exclusão de dados
Resultados esperados	Dados mantidos corretamente após as operações
Dependências	Banco de dados configurado e acesso ao SGBD
PROCEDIMENTOS	
Preparação	Configurar e inicializar o banco de dados
Inicialização	Realizar operações de CRUD
Execução	Verificar consistência dos dados após operações
Recursos	PostgreSql e ferramentas de administração

específicos	
AVALIAÇÃO	
Resultados obtidos	O banco de dados manteve a consistência e integridade dos dados.

5.3 Impacto no desenvolvimento

Os resultados obtidos com o desenvolvimento e a implementação do Integration Hub demonstraram sua eficácia na centralização das integrações com APIs, promovendo melhorias consideráveis nos processos de desenvolvimento e manutenção de software.

A criação de uma interface centralizada para integração com múltiplas APIs gerou benefícios significativos para a empresa, destacando-se:

- Redução do tempo de desenvolvimento: Com o Integration Hub, os desenvolvedores passaram a utilizar APIs já integradas, sem a necessidade de implementações individuais. Antes da implementação, o tempo médio para integração de uma API era de aproximadamente 1 a 2 dias dependendo da complexidade da API, enquanto após a adoção do Integration Hub, esse tempo foi reduzido para poucas horas. Isso acelerou o ciclo de entrega de projetos.
- Simplificação da manutenção: Problemas ou ajustes relacionados às APIs podem ser tratados de maneira centralizada no Integration Hub, eliminando retrabalho e simplificando a evolução do sistema.
- Facilidade de expansão: A arquitetura modular e escalável do sistema permite integrar novas APIs sem comprometer a estrutura existente, garantindo flexibilidade para atender futuras demandas.

6 CONCLUSÃO

O objetivo deste trabalho foi descrever o desenvolvimento de uma solução computacional capaz de realizar a comunicação com outros sistemas, promovendo uma centralização de integrações com vários outros sistemas. Após a fase de execução e realização do projeto, pode-se realizar vários testes de integração e validação, deferindo que a solução atende aos requisitos defendidos, que é eficaz na comunicação com as chamadas a serviços como consultas de CNPJ, CEP e bancos.

A parte principal do projeto foi implementar um sistema central que otimize o processamento e a organização dos dados importados para possibilitar ao usuário final o acesso a informações estruturadas.

O sistema também promove a gestão facilitada das informações ao fornecer uma visão unificada de suas operações e manter um fluxo de dados contínuo e preciso. Dessa forma, o negócio se beneficia, pois passa a contar com mais eficiência e produtividade, já que a pesquisa e a validação de dados demora-se menos e apresenta resultados confiáveis.

A partir dos resultados obtidos, verificou-se que os ganhos operacionais obtidos com a solução estão de acordo com o que a literatura aponta como boas práticas para integração de sistemas. Desta forma, a solução é capaz não apenas de atender as necessidades atuais, mas também atende a capacidade para expansões futuras. Entre as possíveis evoluções, destacam-se: implementação de módulos adicionais e integração de outros sistemas.

7 REFERÊNCIAS

ABIGEE. **The State of APIs: 2016 Report on Impact of APIs on Digital Business.** 2016.

ANUFF, J. **The State of APIs: Report on Impact of APIs on Digital Business.** 2017.

BAVOTA, G.; LINARES-VÁSQUEZ, M.; BERNAL-CÁRDENAS, C.; PENTA, M.; OLIVETO, R.; POSHYVANYK, D. **The impact of API change-and fault-proneness on the user ratings of Android apps.** IEEE Transactions on Software Engineering, v. 41, n. 4, p. 384–407, 2015. doi:10.1109/TSE.2014.2367027.

FELTRIN, L. **APIs REST: o impacto das APIs REST na evolução digital das empresas.** Trabalho de conclusão de curso, Faculdade de Tecnologia de Americana "Ministro Ralph Biasi", 2023.

IBM. **Innovation in the API economy: Building winning experiences and new capabilities to compete.** USA: IBM Institute for Business Value, 2016.

LINARES-VÁSQUEZ, M.; BAVOTA, G.; BERNAL-CÁRDENAS, C.; PENTA, M.; OLIVETO, R.; POSHYVANYK, D. **API change and fault proneness: a threat to the success of Android apps.** In Proceedings of the 2013 9th joint meeting on foundations of software engineering, p. 477-487, 2013. doi:10.1145/2491411.2491428.

MANIKAS, K. **Revisiting software ecosystems research: A longitudinal literature study.** Journal of Systems and Software, v. 117, p. 84–103, 2016. doi:10.1016/j.jss.2016.02.003.

OFOEDA, J.; BOATENG, R.; EFFAH, J. **Application programming interface (API) research: A review of the past to inform the future.** International Journal of Enterprise Information Systems (IJEIS), v. 15, n. 3, p. 76-95, 2019.

SILVA, Marina Aparecida da; SILVA, Douglas da; CAROSIA, Jaciara Silva. T**este de software: um estudo de caso.** VIII Congresso de Trabalhos de Graduação. Faculdade de Tecnologia de Mococa, v. 8, n. 2, 2021.

TAMAE, R.; LIMA, P. **Web Services: Uma Nova Visão Da Arquitetura De Aplicações Distribuídas Na Internet.** Revista.Inf.Br, 2005.

VIGO, Leonardo Parra; OLIVEIRA, Eliane Vendramini de; CAVICHIOLLI, Adriane; AFFONSO, Elaine Parra. **Web services: integração e padronização de serviços.** Revista RETEC, Ourinhos, v. 7, n. 1, p. 36-57, jan./jun. 2014.

MORO, T. D.; DORNELES, C. F.; REBONATTO, M. T. **Web services WS-* versus Web Services REST.** Revista Eletrônica de Iniciação Científica em Computação, v. 11, n. 1, p. 36–51, 2011.

PRESSMAN, R.; MAXIM, B. **Engenharia de Software.** 8. ed. [s.l.] McGraw Hill Brasil, 2016.

LESSA, R. O.; ORIVALDO LESSA JUNIOR, E. **Modelos de Processos de Engenharia de Software.** n. 1, p. 9, 2009.

DA SILVA, R. O.; et al. **O processo de teste de software.** Tecnologias em Projeção, v. 7, n. 2, 2016.

FOWLER, MARTIN. **Padrões de Arquitetura de Aplicações Corporativas.** Tradução de Acauan Fernandes. Porto Alegre: Bookman, 2007.

MACORATTI, JOSÉ CARLOS. **Padrões de projeto: O modelo MVC - Model View Controller.** 1 jun. 2002b. Disponível em: http://www.macoratti.net/vbn_mvc.htm. Acesso em: 8 out. 2024.

BATTIST, JÚLIO. **Criando aplicações em 3, 4 ou n Camadas.** Mai. 2003. Disponível em: http://www.juliobattisti.com.br/artigos/ti/ncamadas.asp. Acesso em: 8 out. 2024.

DOERRFELD, BILL et al. **The API Economy: Disruption and the Business of APIs.** Nordic APIs, 2016.

MAGALHÃES, I. L.; PINHEIRO, W. B. **Gerenciamento de serviços de TI na prática: uma abordagem com base na ITIL.** [s.l.: s.n.].

SOMMERVILLE, I. **Engenharia de Software.** 8. ed. [s.l.] Pearson Addison-Wesley, 2007.

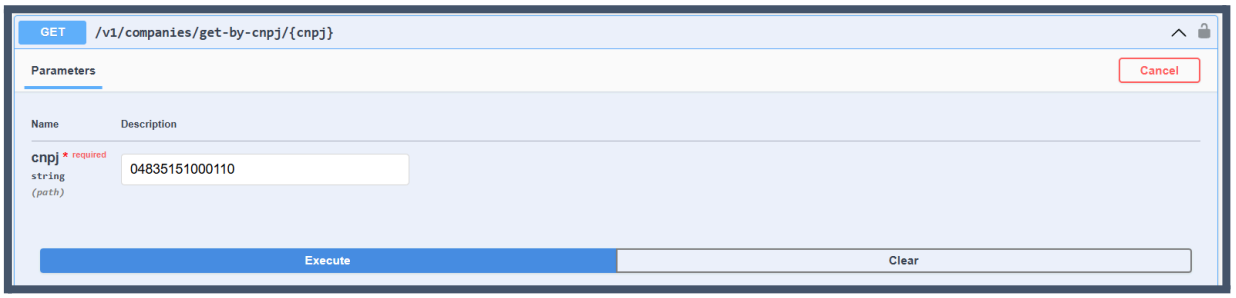
APÊNDICE A - Tabela com o exemplo do serviço de obter por cnpj.

Requisição	<urlserver>/v1/companies/get-by-cnpj?cnpj=”19131243000197”

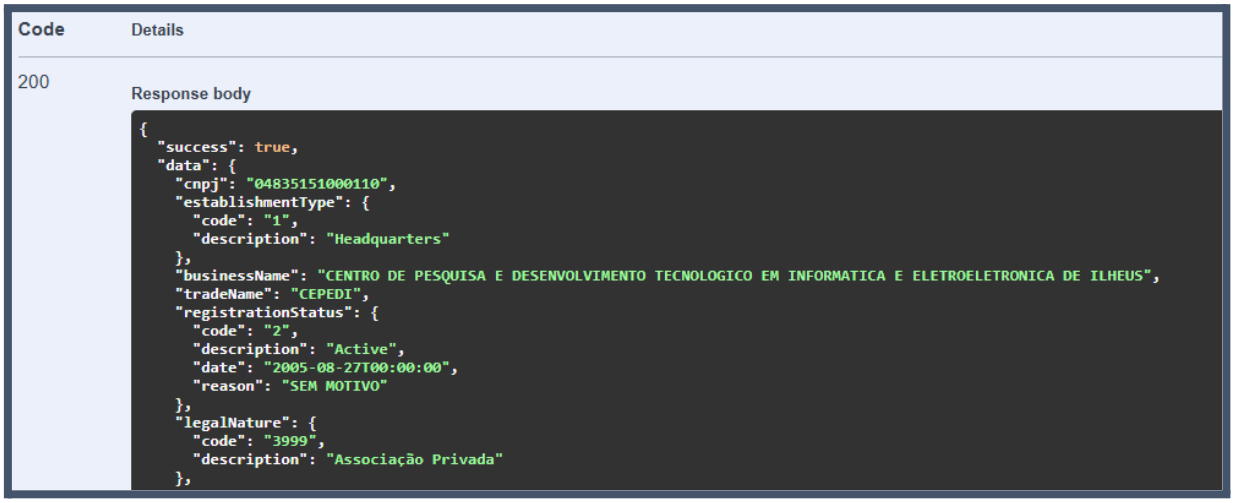
Resposta	{
	"success": true,
	"data": {
	"cnpj": "19131243000197",
	"establishmentType": {
	"code": "1",
	"description": "Headquarters"
	},
	"businessName": "OPEN KNOWLEDGE BRASIL",
	"tradeName": "REDE PELO CONHECIMENTO LIVRE",
	"registrationStatus": {
	"code": "2",
	"description": "Active",
	"date": "2013-10-03T00:00:00",
	"reason": "SEM MOTIVO"
	},
	"legalNature": {
	"code": "3999",
	"description": "Associação Privada"
	},
	"openingDate": "2013-10-03T00:00:00",
	"primaryCnae": {
	"code": "9430800",
	"description": "Atividades de associações de defesa de direitos sociais"
	},
	"secondaryCnaes": [
	{
	"code": "9493600",
	"description": "Atividades de organizações associativas ligadas à cultura e à arte"
	},
	{
	"code": "9499500",
	"description": "Atividades associativas não especificadas anteriormente"
	},
	{
	"code": "8599699",
	"description": "Outras atividades de ensino não especificadas anteriormente"
	},
	{
	"code": "8230001",

```
      "description": "Serviços de organização de feiras,
congressos, exposições e festas"
     },
     {
      "code": "6204000",
      "description": "Consultoria em tecnologia da
informação"
     }
   ],
   "address": {
     "streetType": "AVENIDA",
     "streetName": "PAULISTA 37",
     "number": "37",
     "complement": "ANDAR 4",
     "zipCode": "01311902",
     "neighborhood": "BELA VISTA",
     "city": {
       "code": "3550308",
       "description": "SAO PAULO"
     },
     "country": {
       "code": "105",
       "description": "Brasil"
     },
     "state": "SP"
   },
   "jurisdictionMunicipality": {
     "code": "3550308",
     "description": "SAO PAULO"
   },
   "phones": [
     {
       "areaCode": "11",
       "number": "23851939"
     }
   ],
   "email": null,
   "shareCapital": "0",
   "size": {
     "code": "5",
     "description": "OtherCcompanies"
   },
   "specialStatus": "",
   "specialStatusDate": "2013-10-03T00:00:00"
 }
}
```

APÊNDICE B - Swagger do método de obter por cnpj.



APÊNDICE C - Json com a saída do método de obter por cnpj.



APÊNDICE D - Tabela com o exemplo do serviço de obter por cnpj com parceiros.

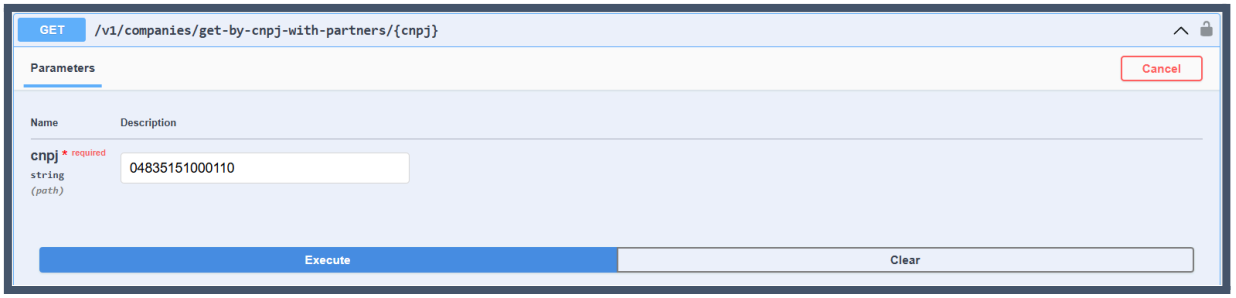
Requisição	<urlserver>/v1/companies/get-by-cnpj-with-partners?cnpj=”19131243000197”
Resposta	{ "success": true, "data": { "cnpj": "19131243000197", "establishmentType": { "code": "1", "description": "Headquarters" }, "businessName": "OPEN KNOWLEDGE BRASIL", "tradeName": "REDE PELO CONHECIMENTO LIVRE", "registrationStatus": { "code": "2",

```
      "description": "Active",
      "date": "2013-10-03T00:00:00",
      "reason": "SEM MOTIVO"
    },
    "legalNature": {
      "code": "3999",
      "description": "Associação Privada"
    },
    "openingDate": "2013-10-03T00:00:00",
    "primaryCnae": {
      "code": "9430800",
      "description": "Atividades de associações de defesa de
direitos sociais"
    },
    "secondaryCnaes": [
      {
        "code": "9493600",
        "description": "Atividades de organizações associativas
ligadas à cultura e à arte"
      },
      {
        "code": "9499500",
        "description": "Atividades associativas não
especificadas anteriormente"
      },
      {
        "code": "8599699",
        "description": "Outras atividades de ensino não
especificadas anteriormente"
      },
      {
        "code": "8230001",
        "description": "Serviços de organização de feiras,
congressos, exposições e festas"
      },
      {
        "code": "6204000",
        "description": "Consultoria em tecnologia da
informação"
      }
    ],
    "address": {
      "streetType": "AVENIDA",
      "streetName": "PAULISTA 37",
      "number": "37",
      "complement": "ANDAR 4",
      "zipCode": "01311902",
      "neighborhood": "BELA VISTA",
      "city": {
```

```
    "code": "3550308",
    "description": "SAO PAULO"
   },
   "country": {
    "code": "105",
    "description": "Brasil"
   },
   "state": "SP"
  },
  "jurisdictionMunicipality": {
   "code": "3550308",
   "description": "SAO PAULO"
  },
  "phones": [
   {
    "areaCode": "11",
    "number": "23851939"
   }
  ],
  "email": null,
  "shareCapital": "0",
  "size": {
   "code": "5",
   "description": "OtherCcompanies"
  },
  "specialStatus": "",
  "specialStatusDate": "2013-10-03T00:00:00",
  "additionalInformation": null,
  "partners": [
   {
    "partnerType": {
     "code": "2",
     "description": "PhysicalPerson"
    },
    "name": "HAYDEE SVAB",
    "qualification": {
     "code": "16",
     "description": "President"
    },
    "inclusionDate": "2024-02-27T00:00:00",
    "legalRepresentative": {
     "name": "",
     "qualification": {
      "code": "0",
      "description": "0"
     }
    }
   }
  ]
```

	} }

APÊNDICE E - Swagger do método de obter por cnpj com parceiros.



APÊNDICE F - Json com a saída do método de obter por cnpj com parceiros.



APÊNDICE G - Tabela com o exemplo do serviço de obter por cnpj com parceiros e seus cpfs.

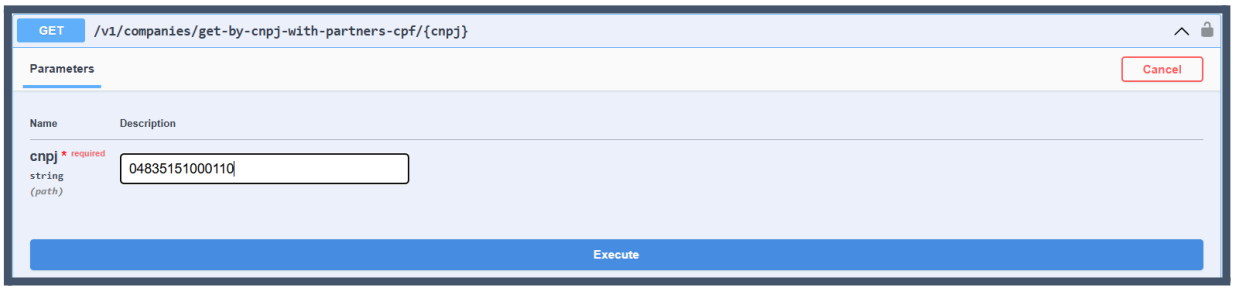
Requisição	<urlserver>/v1/companies/get-by-cnpj-with-partners-cpf?cnpj=”19131243000197”

Resposta	{
	"success": true,
	"data": {
	"cnpj": "19131243000197",
	"establishmentType": {
	"code": "1",
	"description": "Headquarters"
	},
	"businessName": "OPEN KNOWLEDGE BRASIL",
	"tradeName": "REDE PELO CONHECIMENTO LIVRE",
	"registrationStatus": {
	"code": "2",
	"description": "Active",
	"date": "2013-10-03T00:00:00",
	"reason": "SEM MOTIVO"
	},
	"legalNature": {
	"code": "3999",
	"description": "Associação Privada"
	},
	"openingDate": "2013-10-03T00:00:00",
	"primaryCnae": {
	"code": "9430800",
	"description": "Atividades de associações de defesa de direitos sociais"
	},
	"secondaryCnaes": [
	{
	"code": "9493600",
	"description": "Atividades de organizações associativas ligadas à cultura e à arte"
	},
	{
	"code": "9499500",
	"description": "Atividades associativas não especificadas anteriormente"
	},
	{
	"code": "8599699",
	"description": "Outras atividades de ensino não especificadas anteriormente"
	},
	{
	"code": "8230001",
	"description": "Serviços de organização de feiras, congressos, exposições e festas"
	},
	{
	"code": "6204000",

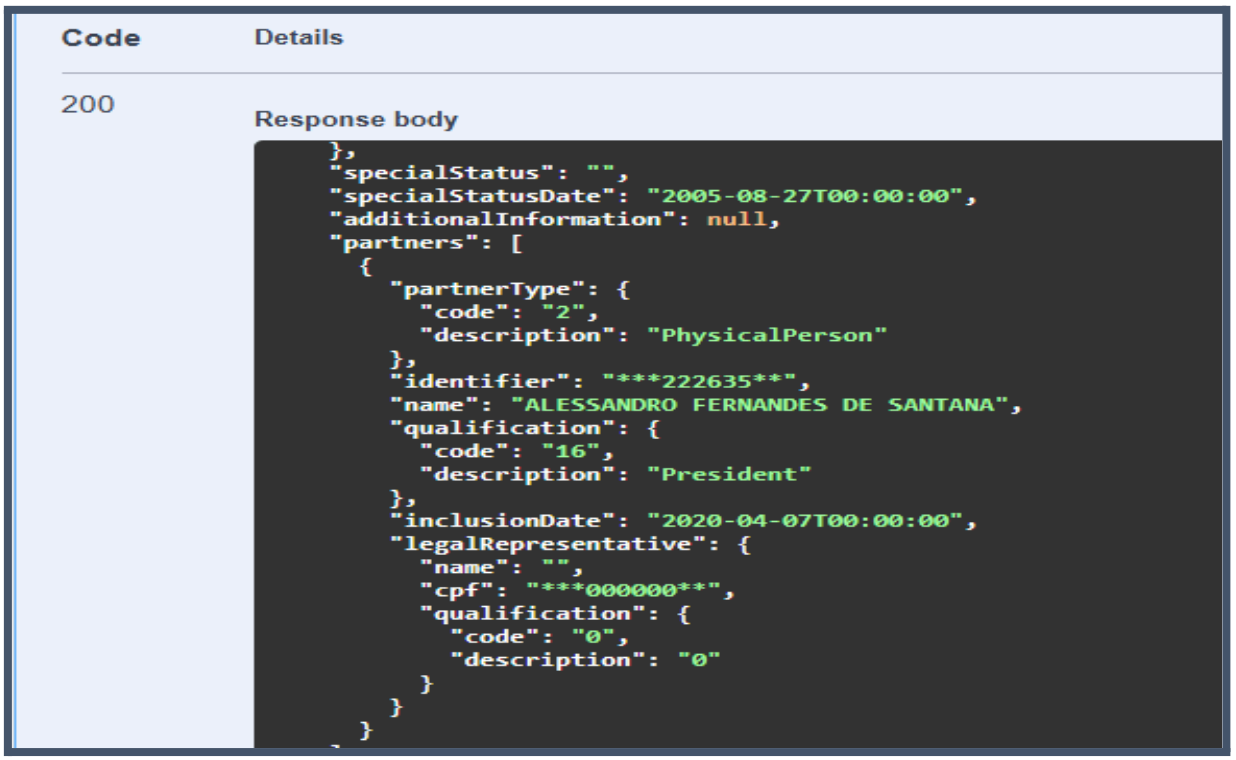
```
      "description": "Consultoria em tecnologia da
informação"
     }
   ],
   "address": {
    "streetType": "AVENIDA",
    "streetName": "PAULISTA 37",
    "number": "37",
    "complement": "ANDAR 4",
    "zipCode": "01311902",
    "neighborhood": "BELA VISTA",
    "city": {
     "code": "3550308",
     "description": "SAO PAULO"
    },
    "country": {
     "code": "105",
     "description": "Brasil"
    },
    "state": "SP"
   },
   "jurisdictionMunicipality": {
    "code": "3550308",
    "description": "SAO PAULO"
   },
   "phones": [
    {
     "areaCode": "11",
     "number": "23851939"
    }
   ],
   "email": null,
   "shareCapital": "0",
   "size": {
    "code": "5",
    "description": "OtherCcompanies"
   },
   "specialStatus": "",
   "specialStatusDate": "2013-10-03T00:00:00",
   "additionalInformation": null,
   "partners": [
    {
     "partnerType": {
      "code": "2",
      "description": "PhysicalPerson"
     },
     "identifier": "***112108**",
     "name": "HAYDEE SVAB",
     "qualification": {
```

```
        "code": "16",
        "description": "President"
      },
      "inclusionDate": "2024-02-27T00:00:00",
      "legalRepresentative": {
        "name": "",
        "cpf": "***000000**",
        "qualification": {
          "code": "0",
          "description": "0"
        }
      }
    }
  ]
}
```


APÊNDICE H - Swagger do método de obter por cnpj com parceiros e seus cpfs.



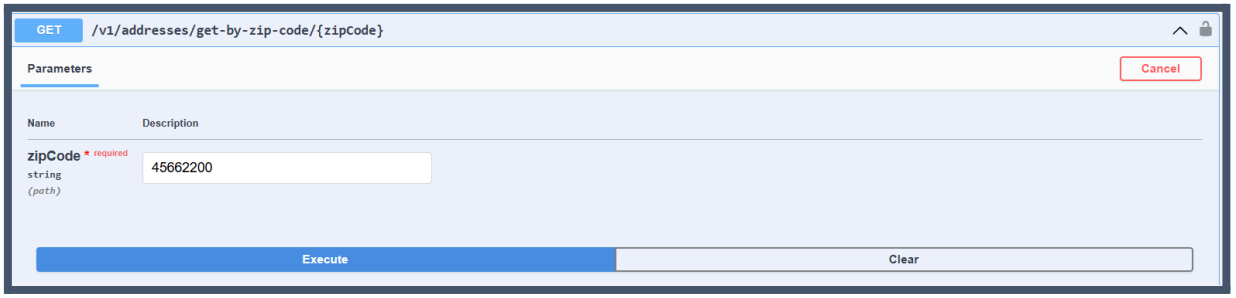
APÊNDICE I - Json com a saída do método de obter por cnpj com parceiros e seus cpfs.



APÊNDICE J - Tabela com o exemplo do serviço de obter pelo cep.

Requisição	<urlserver>/v1/addresses/get-by-zip-code?zipCode=”45662200”
Resposta	{ "success": true, "data": { "zipcode": "45662200", "state": "BA", "city": "Ilhéus", "neighborhood": "Salobrinho", "street": "Rodovia Jorge Amado", "service": "open-cep", "location": null } }

APÊNDICE K - Swagger do método de obter pelo cep.



APÊNDICE L - Json com a saída do método de obter pelo cep.



APÊNDICE M - Tabela com o exemplo do serviço de obter estados.

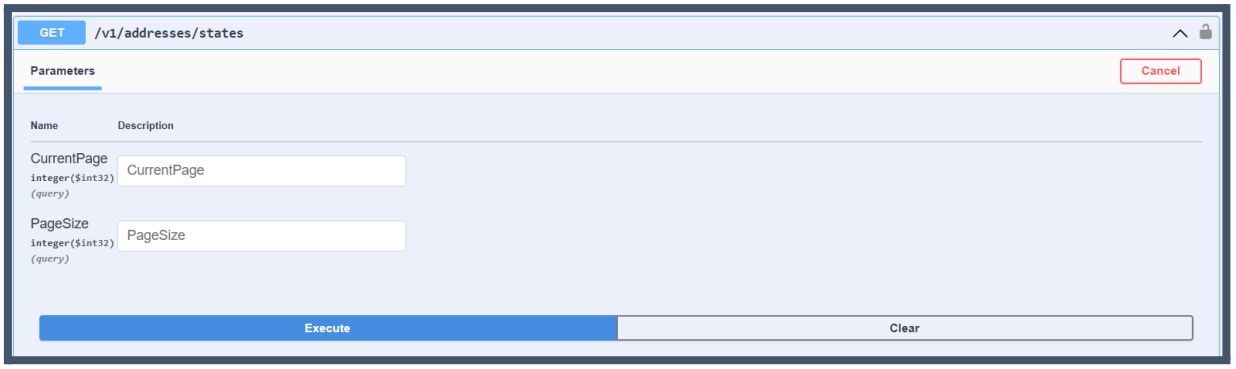
Requisição	<urlserver>/v1/adresses/get-states
Parâmetros de consulta	CurrentPage(int), PageSize(int)

Resposta	{
	"success": true,
	"data": {
	"pagingInformation": {
	"totalCount": 27,

	"totalPages": 3,
	"currentPage": 1,
	"pageSize": 10,
	"hasPrevious": false,
	"hasNext": true
	},
	"items": [
	{
	"code": "AC",
	"name": "Acre"
	},
	{
	"code": "AL",
	"name": "Alagoas"
	},
	{
	"code": "AP",
	"name": "Amapá"
	},
	{
	"code": "AM",
	"name": "Amazonas"
	},
	{
	"code": "BA",
	"name": "Bahia"
	},
	{
	"code": "CE",
	"name": "Ceará"
	},
	{
	"code": "DF",
	"name": "Distrito Federal"
	},
	{
	"code": "ES",
	"name": "Espírito Santo"
	},
	{
	"code": "GO",
	"name": "Goiás"
	},
	{
	"code": "MA",
	"name": "Maranhão"
	}
	]
	}

	}

APÊNDICE N - Swagger do método de obter estados.



APÊNDICE O - Json com a saída do método de obter estados.

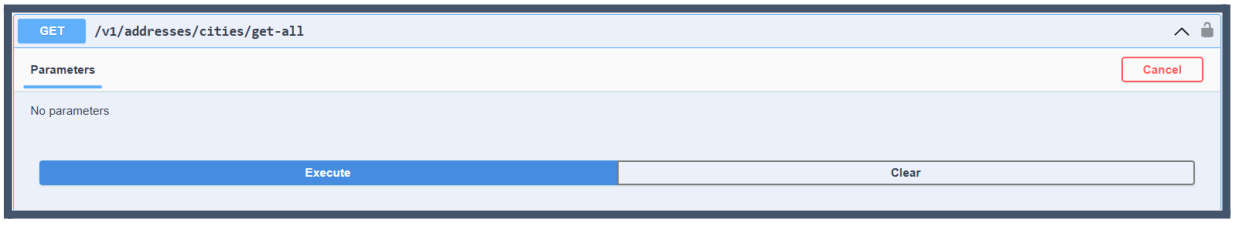


APÊNDICE P - Tabela com o exemplo do serviço de obter todas cidades.

Requisição	<urlserver>/v1/adresses/cities/get-all

Resposta	[
	  {
	  "id": 0,
	  "name": "string",
	  "microregion": {
	  "id": 0,
	  "name": "string",
	  "mesoRegion": {
	  "id": 0,
	  "name": "string",
	  "state": {
	  "code": "string",
	  "name": "string",
	  "id": 0,
	  "region": {
	  "id": 0,
	  "name": "string",
	  "code": "string"
	  }
	  }
	  }
	  }
	  }
	]

APÊNDICE Q - Swagger do método de obter todas cidades.



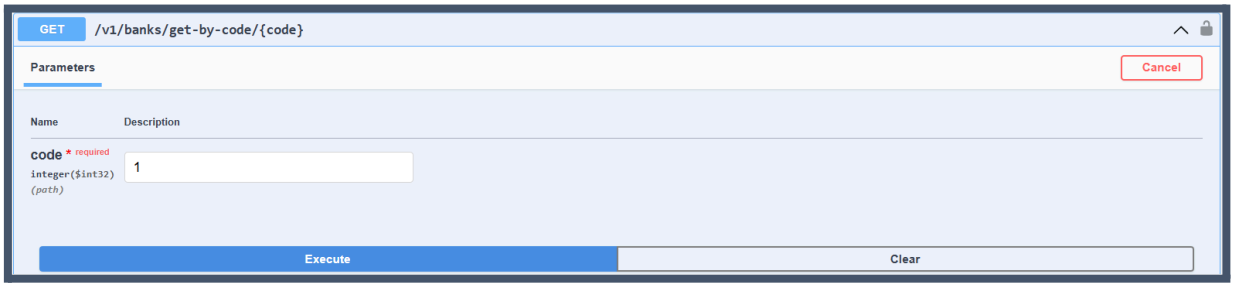
APÊNDICE R - Json com saída do método de obter todas as cidades.



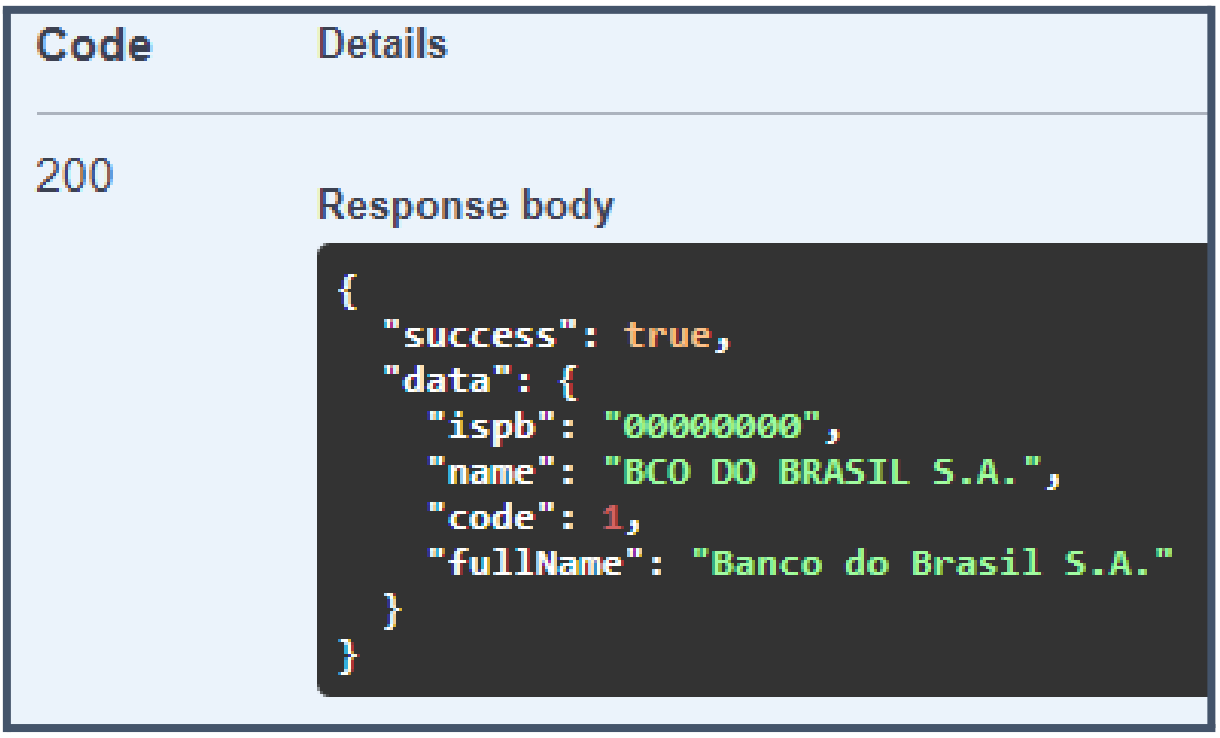
APÊNDICE S - Tabela com o exemplo do serviço de obter bancos por código.

Requisição	<urlserver>/v1/banks/get-by-code?code=1
Resposta	{ "success": true, "data": { "ispb": "00000000", "name": "BCO DO BRASIL S.A.", "code": 1, "fullName": "Banco do Brasil S.A." } }

APÊNDICE T - Swagger do método de obter bancos por código.



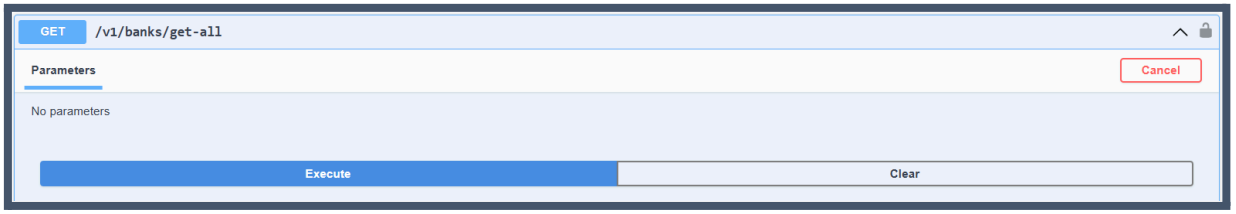
APÊNDICE U - Json com a saída do método de obter bancos por código.



APÊNDICE V - Tabela com o exemplo do serviço todos os bancos.

Requisição	<urlserver>/v1/banks/get-all
Resposta	{ "success": true, "data": [{ "ispb": "00000000", "name": "BCO DO BRASIL S.A.", "code": 1, "fullName": "Banco do Brasil S.A." }, { "ispb": "00000208", "name": "BRB - BCO DE BRASILIA S.A.", "code": 70, "fullName": "BRB - BANCO DE BRASILIA S.A." }, … }

APÊNDICE W - Swagger do método de obter todos os bancos.



APÊNDICE X - Json com saída do método de obter todos os bancos.

